

ỨNG DỤNG PHÂN TÁN

7. Phát triển ứng dụng phân tán

Nhóm chuyên môn UDPT – Trường Công nghệ thông tin Phenikaa
(Trần Đăng Hoan, Đỗ Quốc Trường, Nguyễn Thành Trung, Phạm Kim Thành, Nguyễn Hữu Đạt)

NỘI DUNG

1. Giới thiệu
2. Các thành phần cơ bản cho ứng dụng phân tán
3. Phát triển ứng dụng phân tán với Python RPC
4. Tham khảo thêm: Phát triển ứng dụng phân tán với Java RMI

GIỚI THIỆU

- Ngày nay, Internet đạt đến hàng triệu người trong gần một trăm quốc gia trên tất cả các châu lục trên thế giới
 - ⇒ Ứng dụng thực tế đã chạy trên cả hai hệ thống tập trung và mạng Internet.
 - ⇒ ra đời hệ thống phân tán là không thể tránh khỏi
- Những lợi thế của hệ thống phân tán đã được đưa ra cho mục đích này → phù hợp cho việc phát triển các ứng dụng phân tán.

Các yêu cầu đối với các ứng dụng phân tán

Yêu cầu để xây dựng 1 ứng dụng phân tán đòi hỏi nhiều yếu tố khác nhau, và khá phức tạp. Tuy nhiên, về cơ bản để thiết kế và triển khai một ứng dụng phân tán cần lựa chọn :

- Mô hình phát triển
- Ngôn ngữ lập trình
- Cơ sở dữ liệu
- Kỹ thuật phát triển
- Quản lý bộ nhớ
- Quản lý bảo mật
- ...

NGÔN NGỮ LẬP TRÌNH PHÂN TÁN

- Các ngôn ngữ lập trình cổ điển chẳng hạn như Fortran, Pascal, và C là không thích hợp cho các hệ thống phân tán. Những loại ngôn ngữ này không hỗ trợ tốt cho các vấn đề như:
 - Concurrency
 - Truyền thông
 - Đồng bộ hóa
 - Đáng tin cậy
 - ...

NGÔN NGỮ LẬP TRÌNH PHÂN TÁN

- Hiện có ba vấn đề cơ bản mà phân biệt được chương trình phân tán từ chương trình tuần tự (sequential program)
 - Sử dụng xử lý nhiều yếu tố phân tán (PES)
 - Hợp tác giữa các PES
 - Khả năng chịu lỗi ứng dụng

NGÔN NGỮ LẬP TRÌNH PHÂN TÁN

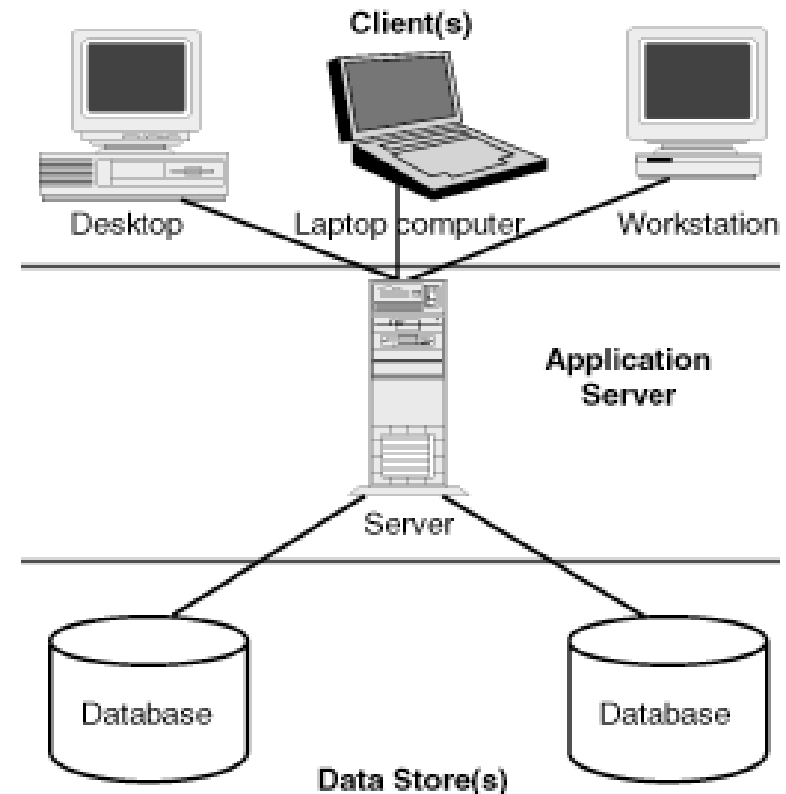
→ Ngôn ngữ hỗ trợ lập trình phân tán có đặc điểm:

- Song song : tiến trình, đối tượng, tường thuật, mapping ...
- Giao tiếp : Thông điệp, chia sẻ dữ liệu, đối tượng...
- Chịu lỗi : trong suốt, giao dịch nguyên tử, lỗi NIL ...

MÔ HÌNH THIẾT KẾ

Client – Server

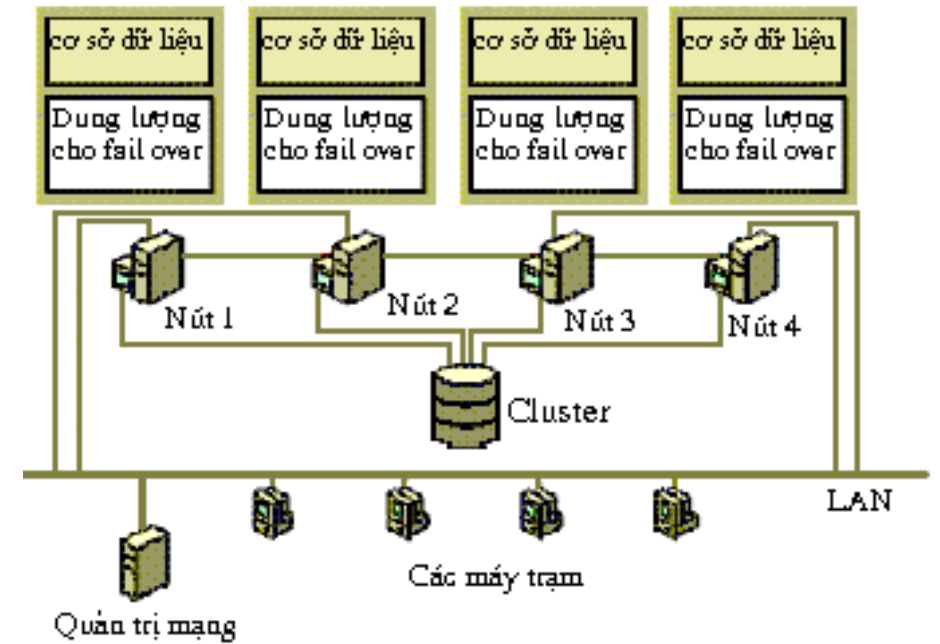
- Là mô hình được áp dụng rất rộng rãi trong các ứng dụng hiện có.
- Trên thực tế thì một server có thể được nối tới nhiều server khác nhằm làm việc hiệu quả và nhanh hơn.
- Khi nhận được 1 yêu cầu từ client, server này có thể gửi tiếp yêu cầu vừa nhận được cho server khác (ví dụ như database server)



Phân theo cụm/đám (Clustering)

Thiết kế và triển khai cluster cần thoả mãn các yêu cầu:

- Tính sẵn sàng cao (availability): tài nguyên luôn sẵn sàng trong khả năng cao nhất để cung cấp, giảm thiểu sự ngưng hoạt động hệ thống
- Độ tin cậy cao (reliability): giảm thiểu tần số xảy ra các sự cố, và nâng cao khả năng chịu đựng lỗi của hệ thống.
- Khả năng mở rộng được (scalability): dễ dàng cho việc nâng cấp, mở rộng trong tương lai.



Phân theo tính toán lưới (Grid Computing)

- Đối với các ứng dụng đòi hỏi: xử lý với số lượng dữ liệu lớn, yêu cầu tìm kiếm, sắp xếp, trích lọc, chia sẻ tài nguyên v.v.. trên môi trường phân tán (chẳng hạn Internet chứa hàng tỷ trang Web) → triển khai Grid Computing.
- Triển khai ứng dụng Grid khác ứng dụng Web.
- Google, Bing, Youtube v.v... sử dụng mô hình này.

Điện toán đám mây

- Điện toán đám mây (cloud computing): mô hình điện toán có khả năng co giãn (scalable) linh động và các tài nguyên thường được ảo hóa được cung cấp như một dịch vụ trên mạng Internet.
- Như vậy, các nguồn điện toán khổng lồ như phần mềm, dịch vụ ... sẽ nằm tại các máy chủ ảo (đám mây) trên Internet. Thay vì trong máy tính gia đình và văn phòng (trên mặt đất) để mọi người kết nối và sử dụng mỗi khi họ cần.

(Tham khảo thêm tại học phần Điện toán đám mây đã học trước đó)

Cơ sở dữ liệu

- Việc lựa chọn CSDL cũng như thiết kế CSDL có 1 ý nghĩa quan trọng trong : vận hành, mở rộng cũng như duy trì, phát triển ứng dụng và xử lý (xử lý đồng thời, tái tạo v.v...) sau này.
- Hiện có 2 mô hình CSDL được triển khai trong các ứng dụng phân tán
 - + **CSDL tập trung**
 - + **CSDL phân tán**

Tùy thuộc vào quy mô của ứng dụng, tính tin cậy, sẵn sàng, hiệu quả v.v... mà chúng ta có thể lựa chọn 1 trong 2 mô hình trên.

KỸ THUẬT PHÂN TÁN

Nhóm 1: Hỗ trợ lập trình thủ tục

- **RPC**, DCE RPC, DCOM

Nhóm 2: Hỗ trợ lập trình hướng đối tượng

- CORBA, **RMI**

Nhóm 3: Hỗ trợ các ứng dụng Web

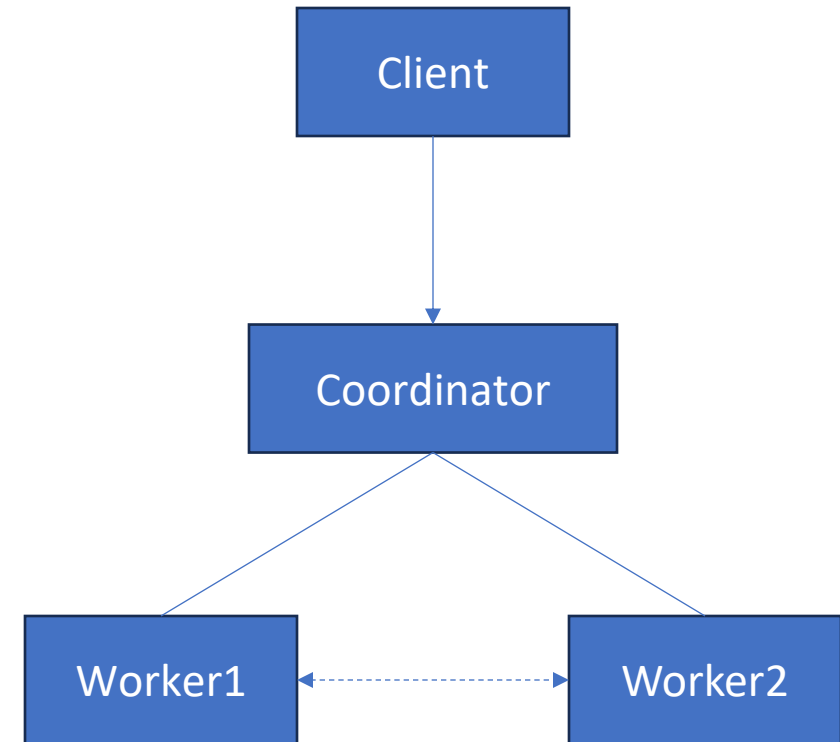
- XML, SOAP, Web Service, AJAX, **REST**

PHÁT TRIỂN UDPT ĐƠN GIẢN VỚI PYTHON RPC

- Bài toán: Hệ thống phân tán với RPC và giao tiếp Worker-Worker
- Mục tiêu:
 - Hiểu cách giao tiếp trong hệ thống phân tán
 - Triển khai giao tiếp Client - Coordinator - Worker bằng RPC
 - Giao tiếp gián tiếp Worker - Worker qua RPC

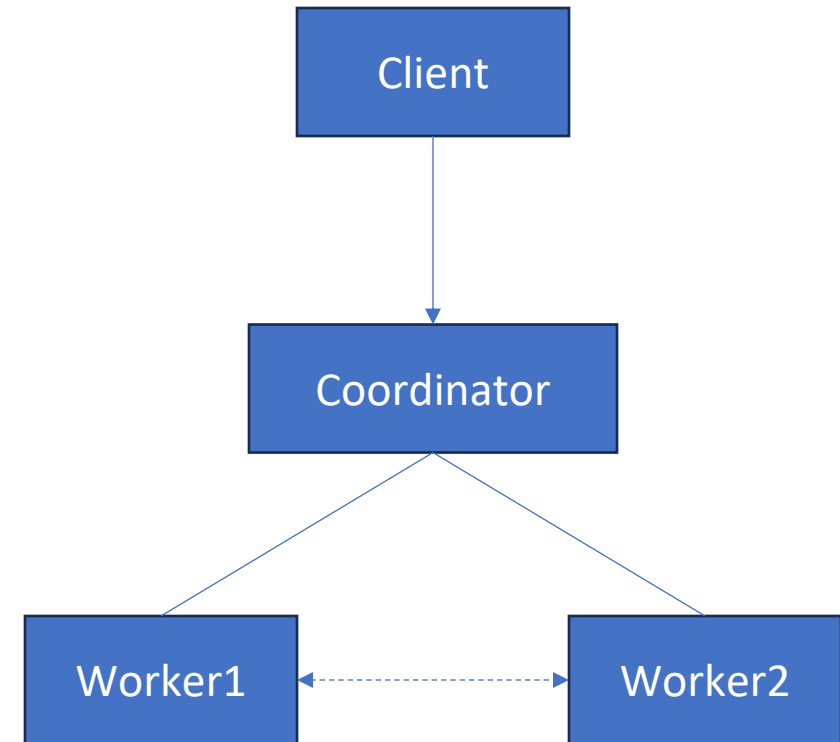
PHÁT TRIỂN UDPT ĐƠN GIẢN VỚI PYTHON RPC

- Bài toán: Hệ thống phân tán với RPC và giao tiếp Worker-Worker
- Ví dụ:
 - Client gửi danh sách số cho Coordinator
 - Coordinator chia dữ liệu và phân công cho Worker
 - Worker1 gửi kết quả trung gian sang Worker2
 - Worker2 tính toán và trả về Coordinator
 - Coordinator trả về Client



PHÁT TRIỂN UDPT ĐƠN GIẢN VỚI PYTHON RPC

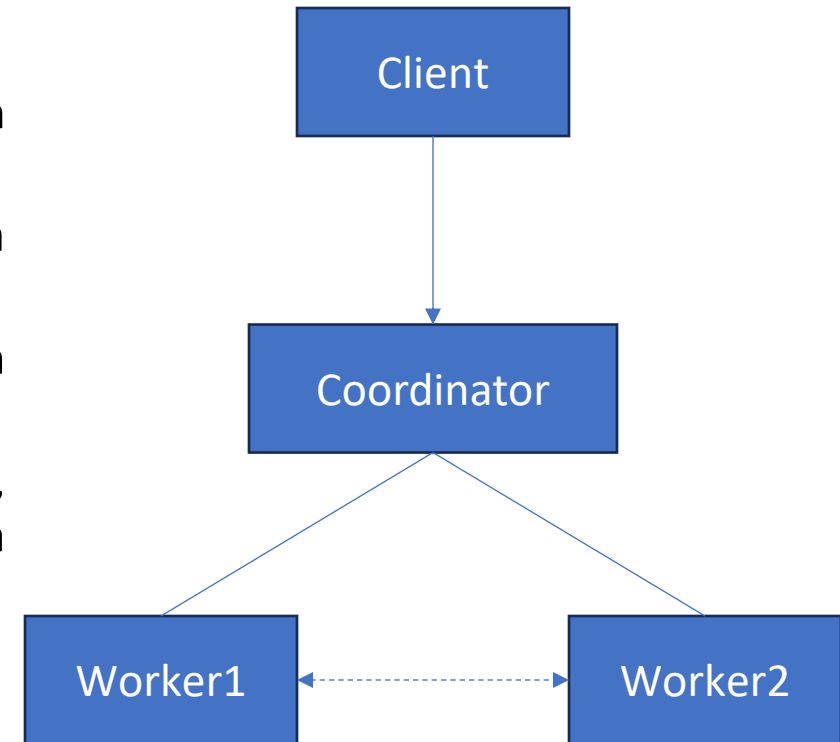
- Bài toán: Hệ thống phân tán với RPC và giao tiếp Worker-Worker
- Ví dụ:
 - Client gửi danh sách số
 - Coordinator chia và cung cấp thông tin Worker
 - Worker1 và Worker2 tính toán và giao tiếp trực tiếp
- Demo: Tính tổng bình phương các số từ 1 đến 10
 - Chia cho 2 worker:
 - Worker1: tính từ 1 đến 5
 - Worker2: tính từ 6 đến 10
 - Worker1 gửi kết quả sang Worker2
 - Worker2 tính tổng cuối cùng



PHÁT TRIỂN UDPT ĐƠN GIẢN VỚI PYTHON RPC

- Demo: Tính tổng bình phương các số từ 1 đến 10
Mã nguồn: (distributed_system_demo)
 - coordinator.py: Chứa thông tin Worker, Cung cấp danh sách IP:PORT cho client và worker
 - worker1.py: Nhận số, tính toán, gửi kết quả qua Worker2 qua RPC
 - worker2.py: Nhận dữ liệu từ Worker1, Tính tổng cả chung và in ra kết quả
 - client.py: Gửi danh sách [1..10] cho Worker1 và Worker2, Worker1 xử lý và gửi kết quả qua Worker2, Worker2 xử lý và trả kết quả cuối

https://drive.google.com/drive/folders/1WNfVOIFk7_eznz2UIZl3or5q9i4CQILN



PHÁT TRIỂN UDPT ĐƠN GIẢN VỚI PYTHON RPC

Bài tập 1: Thêm nhiều Worker (Scalability)

- **Mục tiêu**

- Cho phép hệ thống xử lý dữ liệu lớn hơn bằng cách mở rộng số lượng worker.
- Thể hiện khả năng **chia việc đều**, xử lý song song và gom kết quả.

- **Yêu cầu kỹ thuật**

- coordinator.py: quản lý danh sách n worker.
- client.py: chia dữ liệu đầu vào thành n phần.
- Các worker.py: xử lý phần được phân công, gửi kết quả về cho 1 node gom (hoặc coordinator).

- **SO:**

- Chia công việc trong hệ thống phân tán.
- Vấn đề về cân bằng tải (load balancing).

PHÁT TRIỂN UDPT ĐƠN GIẢN VỚI PYTHON RPC

Bài tập 2: Cho Worker bình chọn Leader (Leader Election)

- **Mục tiêu**

- Giả lập cơ chế **bầu chọn lãnh đạo** trong các hệ thống phân tán (ví dụ: Bully Algorithm).
- Tăng tính tự trị cho hệ thống: không cần coordinator cố định.

- **Yêu cầu kỹ thuật**

- Mỗi worker đều biết danh sách các worker còn lại.
- Khi hệ thống khởi động hoặc khi leader bị mất, các worker thực hiện thuật toán bầu chọn (ví dụ Bully).
- Worker được chọn sẽ thực hiện vai trò leader (gom kết quả, điều phối).

- **SO**

- Leader election (Bully,...)
- Giao tiếp peer-to-peer
- Đồng thuận cơ bản

PHÁT TRIỂN UDPT ĐƠN GIẢN VỚI PYTHON RPC

Bài tập 3: Giao tiếp theo vòng (Ring Topology)

- **Mục tiêu**

- Mô hình hóa hệ thống phân tán theo kiểu token ring.
- Hiểu quy luật truyền thông tuần tự.

- **Yêu cầu kỹ thuật**

- Mỗi worker chỉ biết địa chỉ của worker kế tiếp.
- Dữ liệu/kết quả truyền theo vòng (ví dụ truyền token, hoặc tổng kết dần dần).

- **SO:**

- Mô hình liên kết logic giữa node.
- Truyền thông tuần tự.

THAM KHẢO THÊM

PHÁT TRIỂN UD PHÂN TÁN VỚI JAVA RMI

- Lập trình phân tán đối tượng là một trong những vấn đề nóng bỏng của công nghệ phần mềm hiện nay.
- Làm thế nào để gọi hàm hay đối tượng ở một máy nào đó và gọi chúng từ một máy khác ?

=> Để phát triển các ứng dụng phân tán có thể sử dụng nhiều kỹ thuật khác nhau : RPC, DCE RPC, DCOM, XML, SOAP, Web Service v.v...

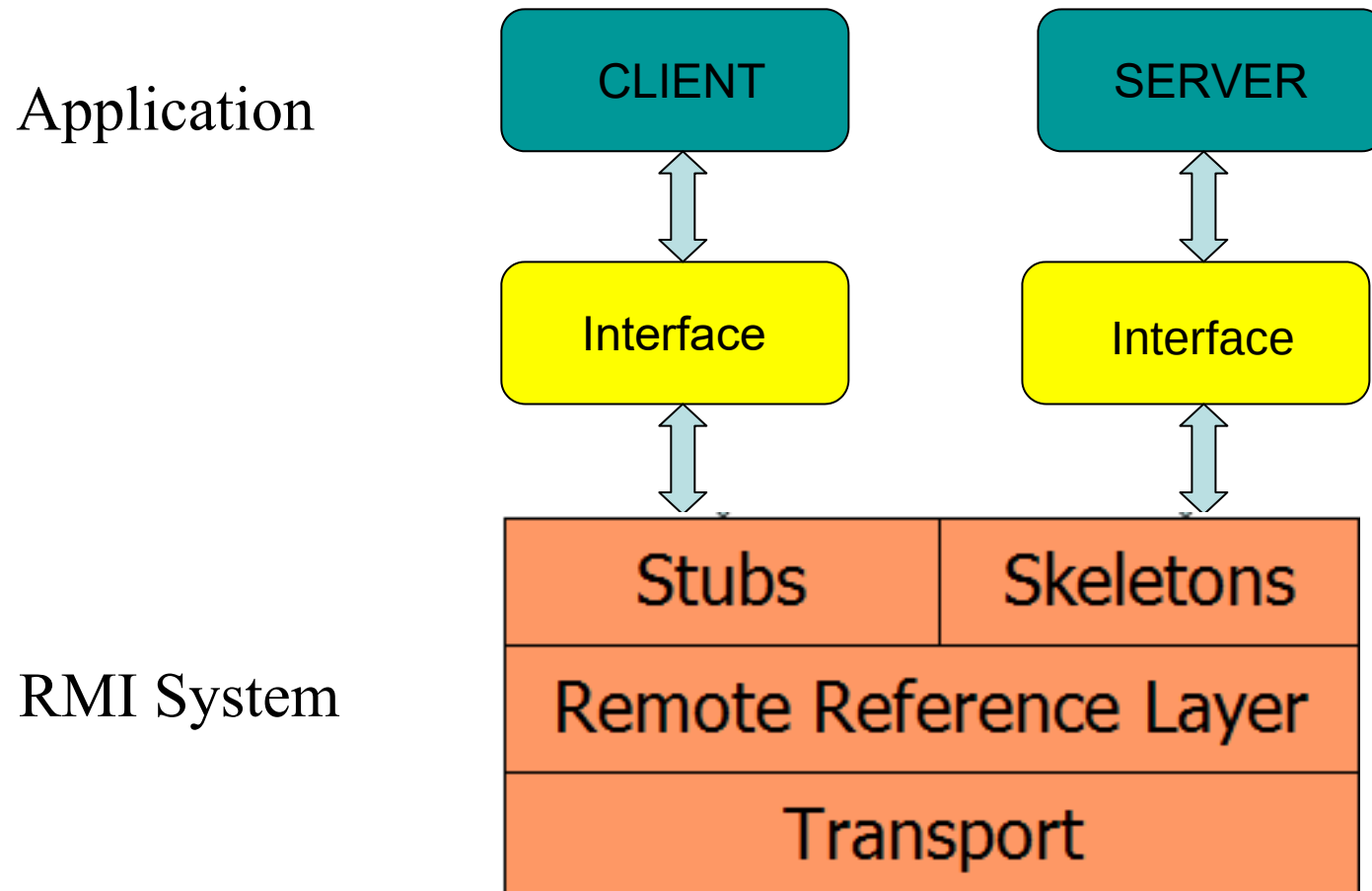
PHÁT TRIỂN UD PHÂN TÁN VỚI JAVA RMI

- Kỹ thuật RMI
 - java.rmi (Tạo interfaces, classes, exceptions ở phía Client)
 - java.rmi.registry (Dịch vụ RMI Naming)
 - java.rmi.server (Tạo interfaces, classes, exceptions ở phía Server)
 - java.rmi.activation (Kích hoạt chức năng theo yêu cầu)
 - java.rmi.dgc (Phân tán, thu gom, giải phóng biến)

PHÁT TRIỂN UD PHÂN TÁN VỚI JAVARMI

- Ưu điểm
 - Đơn giản, dễ sử dụng
 - Trong suốt: lời gọi phương thức từ xa giống lời gọi phương thức cục bộ
 - Độ tin cậy cao
 - An toàn và bảo mật (do JVM cung cấp)
- Nhược điểm:
 - Chỉ dùng cho java

Mô hình tổng quát



Mô hình tổng quát

- Định nghĩa giao diện cho dịch vụ ở xa
- Cài đặt của các dịch vụ ở xa
- Stub và skeleton
- Máy chủ trên đó cài đặt dịch vụ (JRE)
- Dịch vụ tên của RMI để client có thể tìm được dịch vụ ở xa
- Server cung cấp bytecode (HTTP hoặc FTP server)
- Một chương trình client sử dụng dịch vụ ở xa

Quá trình thực hiện

- Tạo 1 lớp giao diện. Ví dụ: *HelloInterface.java*
- Tạo lớp thực hiện mô tả các phương thức của lớp giao diện (Ví dụ: *HelloImplement.java*)
- Xây dựng chương trình Server
 - Tạo đối tượng RemoteObject từ lớp Implement → đăng ký đối tượng với máy JVM.
`UnicastRemoteObject.exportObject(RemoteObject)`
 - Đăng ký đối tượng với rmi registry
`Naming.bind("rmi://<IP address>/tên RemoteObject", RemoteObject);`
- Xây dựng chương trình Client:
 - Tạo một đối tượng Obj tham chiếu đến đối tượng từ xa thông qua:
`Naming.lookup("rmi../tênRemoteObject");`

Quá trình thực hiện

- Biên dịch tạo lớp Stub, Skel :
 - `rmic <tên lớp implement>`
- Biên dịch chương trình Client, Server
- Chạy chương trình
 - chạy `rmiregistry`
 - chạy server
 - chạy client

PHÁT TRIỂN UD PHÂN TÁN VỚI JAVARMI

Ví dụ 1

- Client gửi cho Server lời chào, Server thực hiện trả lời lại cho Client.
- Yêu cầu: Hiển thị lời chào của Server trên màn hình Client theo kỹ thuật RMI

Ví dụ 2

- Client gửi cho Server 1 số nguyên k nhập từ bàn phím đến Server.
- Yêu cầu : Server tính tổng của k số tự nhiên đầu tiên và hiển thị kết quả trên màn hình Client theo kỹ thuật RMI

PHÁT TRIỂN UD PHÂN TÁN VỚI JAVARMI

Ví dụ 1

- B1 : Tạo lớp Interface (Xinchao.java)

```
import java.rmi.*;

//B1: Xây dựng lớp Interface
public interface Xinchao extends Remote {
    public String Hello(String Yourname) throws RemoteException;
}
```

PHÁT TRIỂN UD PHÂN TÁN VỚI JAVARMI

- B2 : Cài đặt lớp thực hiện từ lớp Interface (XinchaoImpl.java)

```
import java.rmi.*;

//B2: Xây dựng lớp thực hiện implements den lop Interface
public class XinchaoImpl extends UnicastRemoteObject implements Xinchao
{
    // Constructre
    public XinchaoImpl() throws RemoteException
    {
        super();
    }
    public String Hello(String Yourname) throws RemoteException
    {
        return "Hi! Chao ban toi ten la: "+Yourname;
    }
}
```

PHÁT TRIỂN UD PHÂN TÁN VỚI JAVARMI

- B3 : Xây dựng đối tượng phân tán trên Server (Server.java)

```
import java.rmi.*;

public class Server{

    public static void main (String[] args){
        try
        {
            //tao doi tuong remote tu phuong thuc XinchaoImpl
            Xinchao h = new XinchaoImpl();

            //dang ky rmi cho doi tuong hva rang buoc vao Server
            Naming.rebind("K15TPM",h);
            System.out.println("Doi tuong tu xa da duoc dang ky");
            System.out.println("Server dang doi yeu cau tu Client!");
        }
        catch(Exception e)
        {
            System.out.println(" Khong the tao duoc doi tuong");
        }
    }
}
```

PHÁT TRIỂN UD PHÂN TÁN VỚI JAVARMI

- B4 : Xây dựng chương trình Client (Client.java)

```
import java.rmi.*;

public class Client{
    public static void main(String[] args)throws Exception
    {
        try
        {
            //tim doi tuong remote trong RMI registry
            Xinchao h = (Xinchao)Naming.lookup("K15TPM");
            System.out.println("Client print: Hello!");
            System.out.println("Server print: "+h.Hello("Lan"));
        }
        catch (Exception e)
        {
            System.out.println(" Không tìm thấy doi tuong tu xa tren Server");
        }
    }
}
```

PHÁT TRIỂN UD PHÂN TÁN VỚI JAVARMI

- B5 : Tạo Stub và Skel (**build.bat**)

```
ECHO OFF
```

```
Echo -----
```

```
Echo          CHUONG TRINH COMPILE
```

```
Echo -----
```

```
Path= C:\Program Files\Java\jdk1.7.0_01\bin
```

```
Echo -----Dich cac tap tin *.java -----
```

```
del *.class
```

```
javac *.java
```

```
Echo ----Tao tap tin trung gian _Stub va _Skel cho doi tuong
```

```
rmic HelloImpl
```

```
start rmiregistry
```

```
@pause
```

PHÁT TRIỂN UD PHÂN TÁN VỚI JAVARMI

- B6 : Chạy chương trình trên server (server.bat)

```
CLS
@ECHO OFF
ECHO =====
ECHO      CUA SO LAM VIEC CUA SERVER
ECHO
=====
Path= C:\Program Files\Java\jdk1.7.0_01\bin
set CLASSPATH=.;%CLASSPATH%;
java Server
pause
```

PHÁT TRIỂN UD PHÂN TÁN VỚI JAVARMI

- B7 : Chạy chương trình trên client (**Client.bat**)

```
CLS
@ECHO OFF
ECHO =====
ECHO      CUA SO LAM VIEC CUA CLIENT
ECHO
=====
Path= C:\Program Files\Java\jdk1.7.0_01\bin
set CLASSPATH=.;%CLASSPATH%;
java Client
pause
```

PHÁT TRIỂN UD PHÂN TÁN VỚI JAVARMI

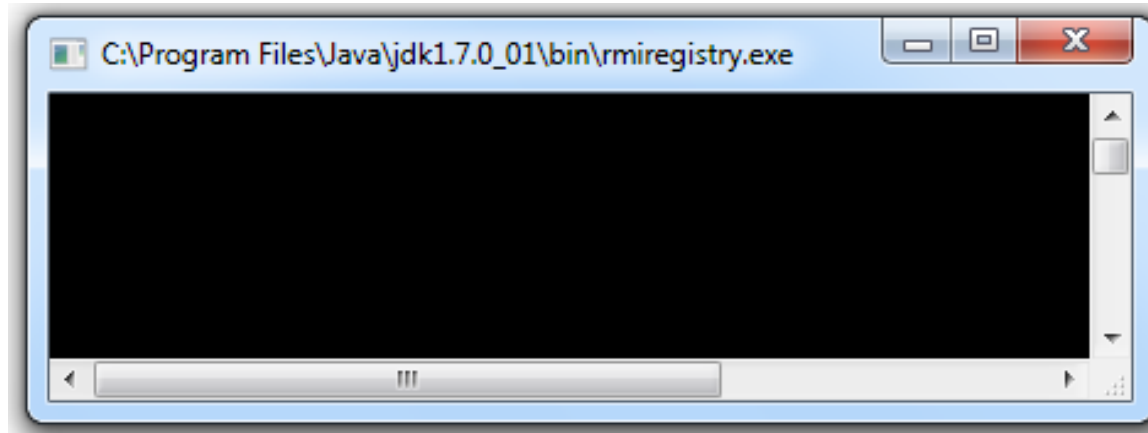
- Tổ chức các lớp tại Client/Server

- Server
 - Server.class
 - Xinchao_Skel.class

- Client
 - Client.class
 - XinchaoImpl.class
 - XinchaoImpl_Stub.class

PHÁT TRIỂN UD PHÂN TÁN VỚI JAVARMI

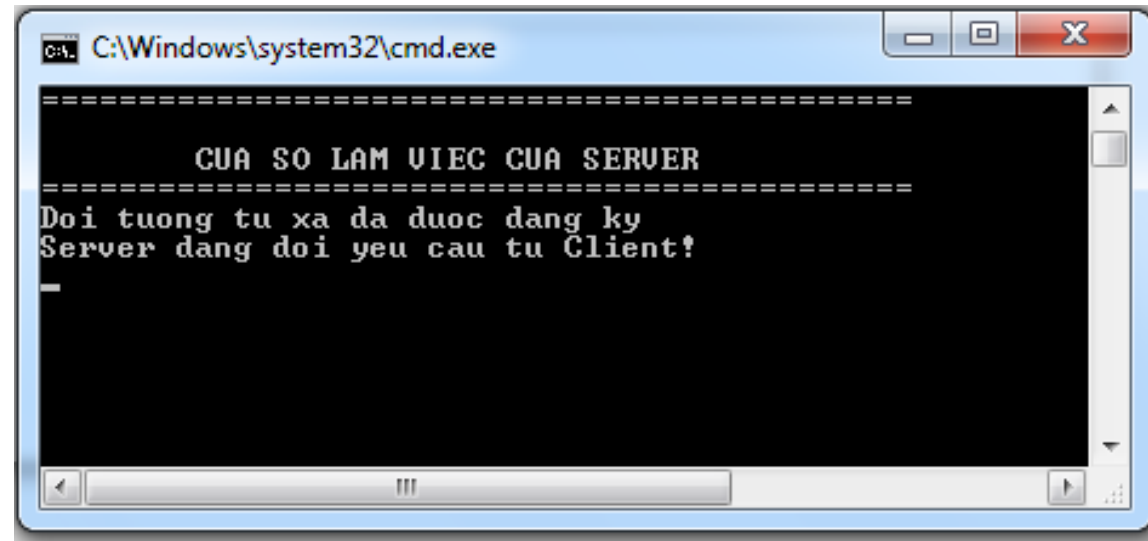
- Lần lượt chạy các file : **build.bat**, **Server.bat**, **Client.bat** ta thu được kết quả như sau :



Kết quả khi chạy file **build.bat** .

PHÁT TRIỂN UD PHÂN TÁN VỚI JAVARMI

- Kết quả khi chạy file **Server.bat**

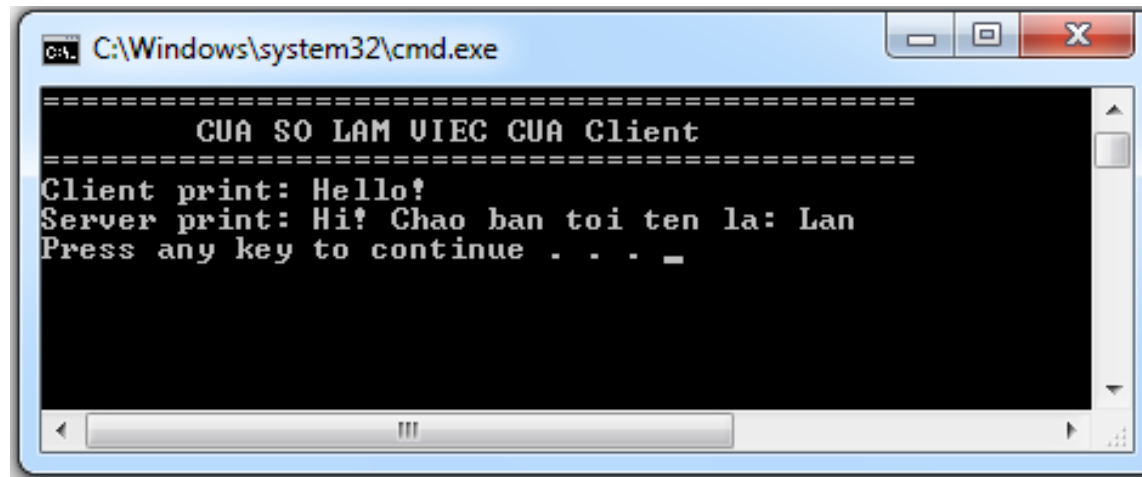


```
C:\Windows\system32\cmd.exe

=====
                CUA SO LAM VIEC CUA SERVER
=====
Doi tuong tu xa da duoc dang ky
Server dang doi yeu cau tu Client!
-
```

PHÁT TRIỂN UD PHÂN TÁN VỚI JAVARMI

- Kết quả khi chạy file **Client.bat**



```
C:\Windows\system32\cmd.exe

=====
          CUA SO LAM VIEC CUA Client
=====
Client print: Hello!
Server print: Hi! Chao ban toi ten la: Lan
Press any key to continue . . . _
```